

05 — Module Genome

Objetivo: describir el ADN de Puglit — los *modules*, los bloques de capacidad reutilizables que el swarm genético VE, REUSA, EXTIENDE y SANA. Si los agentes son las células y el pipeline es el metabolismo, los modules son los genes: unidades discretas de funcionalidad probada que se inyectan de forma determinística en cada app generada y que evolucionan a lo largo de muchos builds.

Todo lo que sigue está verificado contra el código en `/Users/alvaroz/projects/2026/puglit`. Las fuentes principales son `web/lib/module-registry.ts` (el directorio vivo), los ~76 archivos `web/lib/*-module.ts` (las implementaciones determinísticas), `web/lib/swarm-planner.ts` (resolución de dependencias), `web/lib/app-builder.ts` (inyección en `finalize`) y `web/sql/genetic.sql` (la tabla `puglit_modules`). Conteos reales: **85 builtin modules** en `BUILTIN_MODULES` y **76 archivos** `*-module.ts` en `web/lib/`.

1. What Is a Module

Un *module* es una **unidad de capacidad reutilizable** que sabe cómo agregarse a una app full-stack Next.js 16 + Postgres sin que el codegen genérico tenga que reinventarla. La diferencia clave con un "snippet" o una librería npm es que un module de Puglit no es solo código: es código + **metadata** + **semántica** + **reglas de inyección** + **dependencias** + **SQL**, todo empaquetado para que el swarm pueda razonar sobre él.

El contrato vive en `web/lib/module-registry.ts`:

```
export interface Module {
  name: string
  category: "channel" | "integration" | "util" | "agent"
  description: string
  whenToUse?: string
  envVars?: string[]
  deps?: Record<string, string>
  gateway?: string
  files?: ModuleFile[]
  version?: number
  status?: string // experimental | candidate | stable | core
  requires?: string[] // dependencias duras de otros modules
  tier?: "core" | "infra" | "integration" | "business" | "meta"
}
```

Hay dos grandes familias de modules, que conviven en el mismo registry pero se materializan distinto:

- **Builtin modules** — su metadata está catalogada en `BUILTIN_MODULES` (85 entradas), pero su **código real vive en archivos** `web/lib/*-module.ts` (76 archivos) o en `connectors.ts` / `integrations.ts`. Cada uno exporta una función `deterministicX(config, bp)` que decide,

mirando el blueprint, si la app necesita esa capacidad y, en caso afirmativo, devuelve `{ files, extraSql }`.

- **Custom modules** (harvested) — los que el propio swarm escribió en builds pasados. Viven en Postgres (`puglit_modules`) y se espejan a `modules/<name>/` para revisión en git.

El principio que define la genética: el module **se adapta al esquema detectado, nunca asume nombres**. El ejemplo canónico es `rentals-module.ts`. Su función `detectTables(bp)` mira las columnas y nombres de tabla del blueprint del arquitecto y descubre cómo se llama la tabla de reservas (`bookings`, `reservations`, `reservas`, `estancias` ...), parametrizando ese nombre en todos los primitivos, el SQL, el override de ruta y los tests:

```
book: byCol(/^check_in$/i) || byName(/booking|reserv|stay|estan/i, "bookings"),
```

Esto es **NO HARDCODING** llevado al extremo: la lógica de dominio (pricing determinístico, disponibilidad half-open, anti-double-booking atómico) está pre-construida y testeada, pero el binding a la tabla concreta se resuelve en runtime de generación. El module es un gen; la app es la expresión de ese gen contra un fenotipo (esquema) particular.

Por qué inyección determinística y no "que lo escriba el LLM"

La lógica correctness-critical (pricing en centavos enteros donde `search == checkout`, una constraint `EXCLUDE USING gist` para que dos reservas que se solapan no puedan coexistir, refunds por snapshot de política) es exactamente el tipo de cosa que un LLM escribe sutilmente mal. Puglit la encapsula una sola vez, la testea con vitest real, y la inyecta byte-por-byte cuando el dominio matchea. El LLM no la reescribe; la **reusa**.

2. Module Lifecycle (Experimental → Candidate → Stable → Core → Deprecated)

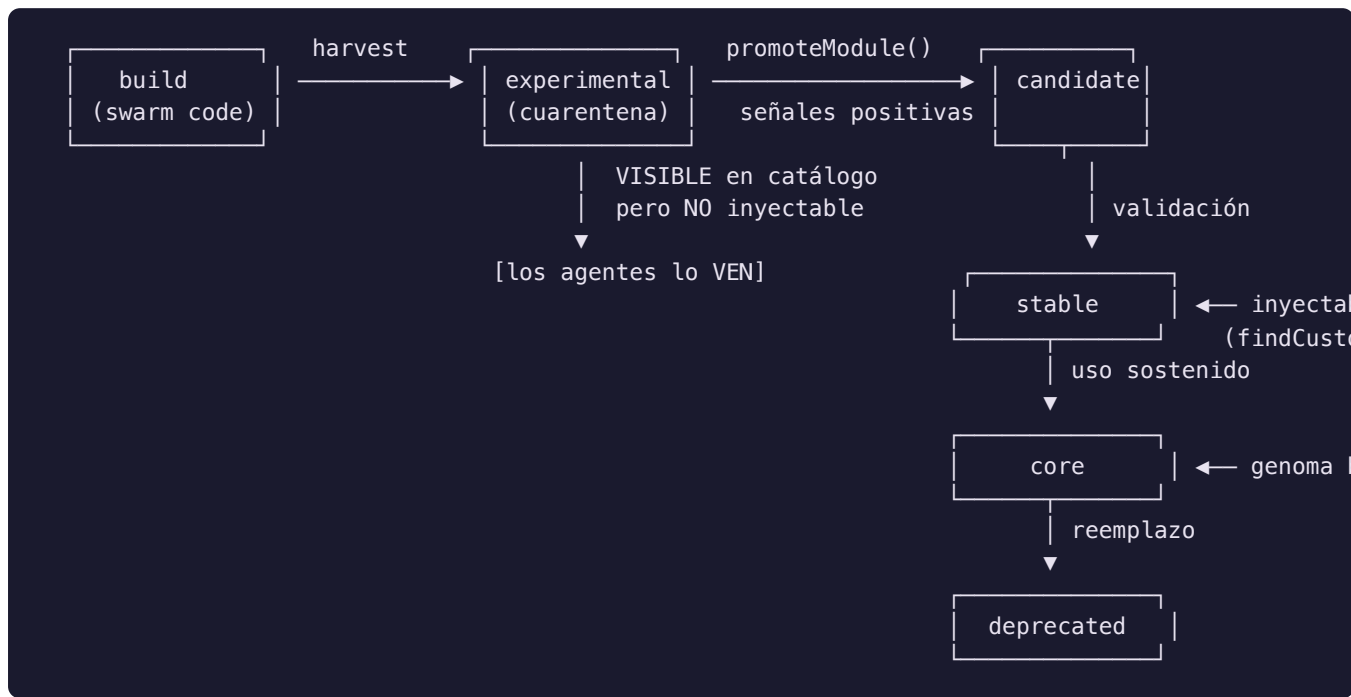
El registry es **vivo**: no es un catálogo fijo que un humano cura, sino un organismo que crece con cada build. El campo `status` define la posición de cada module en su ciclo de vida.

Stage	Significado	¿Auto-inyectado en builds reales?	Confianza
experimental	Recién harvested de un build. Código sin curar.	No	Baja — cuarentena
candidate	Promovido tras señales positivas. En evaluación.	No	Media
stable	Probado en múltiples builds, confiable.	Sí	Alta
core	Capacidad fundacional, parte del genoma base.	Sí	Máxima
deprecated	Reemplazado o roto. Se mantiene por trazabilidad.	No	—

La regla de gobernanza está codificada explícitamente en `findCustomModulesFor()`:

```
if (!["stable", "core"].includes(m.status || "")) return false
// experimental/candidate stay out
```

Esto es **harvest governance**: un module recién cosechado (**experimental**) es visible para los agentes en el catálogo (lo VEN, aprenden de su existencia) pero **no contamina builds reales** hasta que cruza el umbral a **stable**. Previene el *catalog rot* — que código a medio hornear se filtre a productos de cliente.



La promoción es una transición de un solo campo, implementada en **promoteModule()**:

```
export async function promoteModule(name: string, to: "candidate" | "stable" | "core"): Promise<void> {
  await query("UPDATE puglit_modules SET status=$2, updated_at=NOW() WHERE name=$1", [name, to])
}
```

Nótese que los **85 builtins NO tienen status explícito en su metadata** — son **stable / core** por construcción (su código está versionado en el repo y testado). El lifecycle aplica con todo su rigor a los *modules harvested*, que son los que entran "crudos" desde la generación.

3. Module Schema (Metadata · Inputs · Outputs)

El schema de un module tiene tres planos: la **metadata** (lo que el swarm razona), los **inputs** (lo que necesita para activarse) y los **outputs** (lo que produce).

Metadata

Es la parte que se inyecta en los prompts de los agentes vía **moduleCatalog()**. Cada module se serializa en una línea legible:

```
return mods.map((m) =>
  `-${m.name} (${m.category}): ${m.description}` +
  `${m.whenToUse ? ` - use when ${m.whenToUse}` : ""}` +
  `${m.gateway ? ` [needs ${m.gateway}]` : ""}`
).join("\n")
```

Campos relevantes:

- `name` — identificador único (clave primaria en `puglit_modules`).
- `category` — una de cuatro: `channel`, `integration`, `util`, `agent`.
- `description` — qué hace, en prosa densa orientada al LLM.
- `whenToUse` — la heurística de activación humana ("the product charges money / subscriptions / e-commerce").
- `gateway` — el sidecar Docker que necesita, si lo hay (p.ej. ScrapeGraphAI, n8n, Nango, MinIO, Keycloak...).

Inputs

- `envVars` — las variables de entorno que la app generada necesitará. Esto es BYO-credentials: Puglit nunca embebe secretos, solo declara qué llaves harán falta (`STRIPE_SECRET_KEY`, `ELEVENLABS_API_KEY`, `S3_ENDPOINT` ...).
- `config + bp` — los argumentos de la función `deterministicX(config: DomainConfig, bp: Blueprint)`. El module inspecciona `config.identity`, `bp.summary`, `bp.tables` para decidir si se activa. Ejemplo de `wallet-module.ts`:

```
const hay = `${config.identity.name} ${bp.summary} ${bp.tables.map((t) => t.name).join(" ")}`
const wants = /credit|cr[eé]dito|wallet|billetera|points|puntos|token|saldo|balance|coin|moneda/
if (!wants) return null
```

El regex bilingüe (español/inglés) refleja que Puglit nació LATAM-first.

Outputs

Cada module devuelve `{ files: ModuleFile[]; extraSql: string }` (o `null` si no aplica):

- `files` — uno o más `{ path, content }`. Para `rag-module.ts` es `lib/rag.ts`; para `rentals-module.ts` son siete archivos (`pricing.ts`, `availability.ts`, `booking.ts`, `refund.ts`, `reviews.ts`, un test domain `__tests__/domain.test.ts`, y `app/api/health/route.ts`).
- `extraSql` — el DDL que se concatena a `sql/app.sql`. Aquí vive lo más valioso: la `EXCLUDE USING gist` de rentals, el `CREATE EXTENSION vector` de rag, el ledger append-only de wallet.

El SQL es siempre **idempotente y guardado**. Ejemplo de rag-module, que no debe tirar abajo todo el `app.sql` si pgvector no está instalado:

```
DO $$ BEGIN
  CREATE EXTENSION IF NOT EXISTS vector;
  CREATE TABLE IF NOT EXISTS rag_documents (...);
EXCEPTION WHEN OTHERS THEN RAISE NOTICE 'pgvector unavailable - RAG store skipped'; END $$;
```

4. Dependency Rules (`MODULE_REQUIRES` , `dependencyClosure`)

Un module rara vez vive solo. `payments` necesita guardar tokens cifrados → necesita `crypto` . `rag` necesita un cliente de embeddings → necesita `llm` . Inyectar un dependiente sin su base produce un build roto. Puglit codifica esto como un **grafo de dependencias duras** en `MODULE_REQUIRES` :

```
export const MODULE_REQUIRES: Record<string, string[]> = {
  "social-auth": ["crypto"],
  billing: ["crypto"],
  payments: ["crypto"],
  inappnotify: ["realtime"],
  moderation: ["llm"],
  rag: ["llm"],
  agent: ["llm", "crypto"],
  charts: ["stats"],
  forms: ["validation"],
  webhooksout: ["crypto", "queue"],
  imagegen: ["storage"],
  media: ["storage"],
  graphify: ["llm"],
}
```

Trece relaciones declaradas. La resolución es una **clausura transitiva** clásica (DFS sobre el grafo):

```
export function dependencyClosure(names: string[]): string[] {
  const seen = new Set<string>(names)
  const stack = [...names]
  while (stack.length) {
    const n = stack.pop()!
    for (const dep of MODULE_REQUIRES[n] || []) if (!seen.has(dep)) { seen.add(dep); stack.push(dep) }
  }
  return [...seen]
}
```

Garantiza que si un keyword activa `agent` , la clausura arrastra `llm` y `crypto` ; si activa `webhooksout` , arrastra `crypto` y `queue` . No hay forma de shippear un dependiente sin su base.

Esto se aplica en dos lugares:

1. `swarm-planner.ts` usa `dependencyClosure(present)` durante la planificación de capacidades, para que el plan ya contemple las bases.
2. `resolveDeps(files)` en el planner recorre `MODULE_REQUIRES` , detecta qué modules quedaron presentes mirando sus archivos primarios (mapa `PRIMARY` , p.ej. `payments` → `lib/payments.ts`), corre la clausura, y fuerza la inyección de cualquier base faltante vía `DEP_INJECTORS` , agregando el SQL con un comentario `-- auto-dependency: <dep>` . El binding es por **archivo presente**, no por declaración — si el código de un module está, su dependencia se materializa.

5. Versioning

Cada module lleva un `version: number` . La versión arranca en `1` y se incrementa **monótonamente en cada mejora** (un upsert). La lógica está en `registerModule()` :

```
INSERT INTO puglit_modules (...) VALUES ($1,...,1,$9,$10)
ON CONFLICT (name) DO UPDATE SET
  ...,
  version = puglit_modules.version + 1,
  status = 'improved',
  updated_at = NOW()
```

Tres consecuencias de diseño:

- **Upsert por name** — el nombre es la clave primaria (`puglit_modules.name VARCHAR(64) PRIMARY KEY`). Un module nuevo inserta con `version=1`; reescribir uno existente bumpea la versión y marca `status='improved'`.
- **Self-healing** — cuando un build produce una versión *mejor* de un connector existente, el upsert hace fluir la mejora de vuelta al registry. El comentario en el código lo llama explícitamente "Agents HEAL it".
- **Override de builtin** — en `allModules()`, un custom module con el mismo nombre que un builtin **sobreescribe al builtin**: "a custom module overrides a builtin of the same name = an improvement". El genoma evolucionado le gana al genoma de fábrica.

```
export async function allModules(): Promise<Module[]> {
  const custom = await customModules()
  const overridden = new Set(custom.map((m) => m.name))
  return [...BUILTIN_MODULES.filter((m) => !overridden.has(m.name)), ...custom]
}
```

El espejo a git (`modules/<name>/module.json` + los archivos) hace que cada versión sea **revisable y diffeable** por humanos, aunque la fuente de verdad sea Postgres.

6. Harvest Process

El *harvest* es el mecanismo por el cual el directorio **CRECE desde el trabajo del propio swarm**. Tras cada build, `harvestModules(files, createdBy)` escanea los archivos generados buscando connectors/integraciones reusables que aún no estén catalogados:

```
const m = f.path.match(/^lib\/(connectors|integrations)\/([a-z0-9_-]+)\.ts$/i)
if (!m) continue
const name = m[2].toLowerCase()
if (name === "types" || name === "index" || BUILTIN_NAMES.has(name)) continue
if ((await customModules()).some((x) => x.name === name)) continue
await registerModule({
  name, category: m[1] === "connectors" ? "channel" : "integration",
  description: `Auto-harvested ${...} connector from a generated app.`,
  whenToUse: `the product uses ${name}`, files: [f], status: "experimental", createdBy,
})
```

Reglas del harvest:

1. **Patrón de path estricto** — solo cosecha archivos en `lib/connectors/*.ts` o `lib/integrations/*.ts`. Es una convención: si un agente escribe un connector reusable, lo pone ahí.

2. **Exclusiones** — ignora `types`, `index` y cualquier nombre que ya sea builtin o ya esté en custom. No duplica.
3. **Categoría inferida** — `connectors/` → `channel`; `integrations/` → `integration`.
4. **Status inicial** = `experimental` — todo lo cosechado entra en **cuarentena**. Visible en el catálogo, pero fuera de los builds reales hasta promoción.
5. `createdBy = bp.kind` — se registra qué tipo de proyecto lo originó, para trazabilidad.

El harvest se llama en `finalize`, justo después de inyectar los custom modules relevantes:

```
// app-builder.ts ~línea 1503
await harvestModules(files, bp.kind).catch(() => {})
```

Es **fire-and-forget tolerante a fallos** (`.catch(() => {})`): si el harvest falla, el build no se cae. El catálogo creciendo es un *nice-to-have*, no un bloqueante.

Este es el bucle evolutivo central: **catalog** → **harvest** → **promote**. Los agentes VEN el catálogo (moduleCatalog en sus prompts) → escriben código nuevo cuando algo no existe → ese código se cosecha → tras validación se promueve → pasa a estar disponible para *todos* los proyectos futuros. El genoma de Puglit se expande solo.

7. Repair Process

El "repair" de modules tiene dos caras complementarias.

Self-healing por versión (el camino feliz). Como se vio en §5, cuando un build produce una versión mejorada de un module existente, el upsert con bump de versión hace que la mejora fluya de vuelta. No hay un "fixer" separado; el mecanismo de registro es el de reparación. Un fix a un connector en el build N queda disponible para el build N+1.

Robustez del SQL de salida. Cada module está escrito para **no romper la app aunque su infraestructura falte**. Es reparación preventiva en el output:

- `rag-module.ts` envuelve el `CREATE EXTENSION vector` en un bloque `DO $$ ... EXCEPTION WHEN OTHERS THEN RAISE NOTICE ... END $$;` — si pgvector no está, el RAG store se saltea y el resto del esquema carga.
- `rentals-module.ts` envuelve sus `ALTER TABLE` y la constraint `EXCLUDE` en bloques con `EXCEPTION WHEN duplicate_object ... WHEN undefined_table ... WHEN undefined_column THEN NULL` — idempotente y a prueba de orden de ejecución.

Anti-double-booking a nivel de base. El `createBooking` de rentals usa una transacción + la constraint de exclusión, y traduce el error Postgres `23P01` a un HTTP 409 limpio:

```
if (e.code === "23P01" || /no_double_booking|exclu/i.test(e.message || "")) throw new BookingError
```

La invariante de negocio (no dos reservas solapadas) la garantiza la base, no el código de aplicación; el module solo la traduce a una respuesta correcta. El **runtime gate** del whitepaper (boot real + smoke test)

es la red de seguridad que detecta cuando un module mal inyectado produciría un 5xx — el caso Stayforge expuso 7 bugs reales invisibles a un compile verde.

8. Conflict Resolution

Cuando dos fuentes quieren producir el mismo archivo o la misma capacidad, Puglit resuelve con reglas deterministas, no con azar.

Builtin vs custom (mismo `name`). El custom gana. `allModules()` filtra el builtin homónimo y deja al custom. Filosofía: lo que el swarm aprendió supera lo de fábrica.

Colisión de archivos en `finalize`. El patrón es uniforme en todo `app-builder.ts`: **first-writer-wins por path**. Cada inyección chequea presencia antes de empujar:

```
for (const f of cm.files || []) if (!files.some((x) => x.path === f.path)) files.push(f)
```

El orden de inyección define la precedencia. Esto importa para los **route overrides**:

`deterministicRentalRoutes` *reemplaza* la ruta de creación de booking que el swarm haya escrito, porque la versión verificada del module debe ganarle a la versión libre del LLM. Localiza el candidato existente por regex y lo sobrescribe:

```
const re = new RegExp(`^app/api/${T.book}|bookings|reservations|reservas)/route\\.ts$`)
const cand = existingPaths.find((p) => re.test(p))
return [{ path: cand || `app/api/${T.book}/route.ts`, content: bookingRoute(T) }]
```

Dependencias vs presencia. `resolveDeps` solo inyecta una base si su archivo primario no existe ya (`if (!inj || has(...)) continue`), y solo agrega su SQL si el `app.sql` no lo contiene ya (chequea los primeros 40 chars del fragmento). No hay doble inyección.

9. Compatibility Matrix

La "matriz de compatibilidad" de Puglit es, operativamente, la combinación de `category` + `MODULE_REQUIRES` + `gateway`. Define qué encaja con qué y qué infraestructura externa se arrastra.

Pares de dependencia verificados (de `MODULE_REQUIRES`):

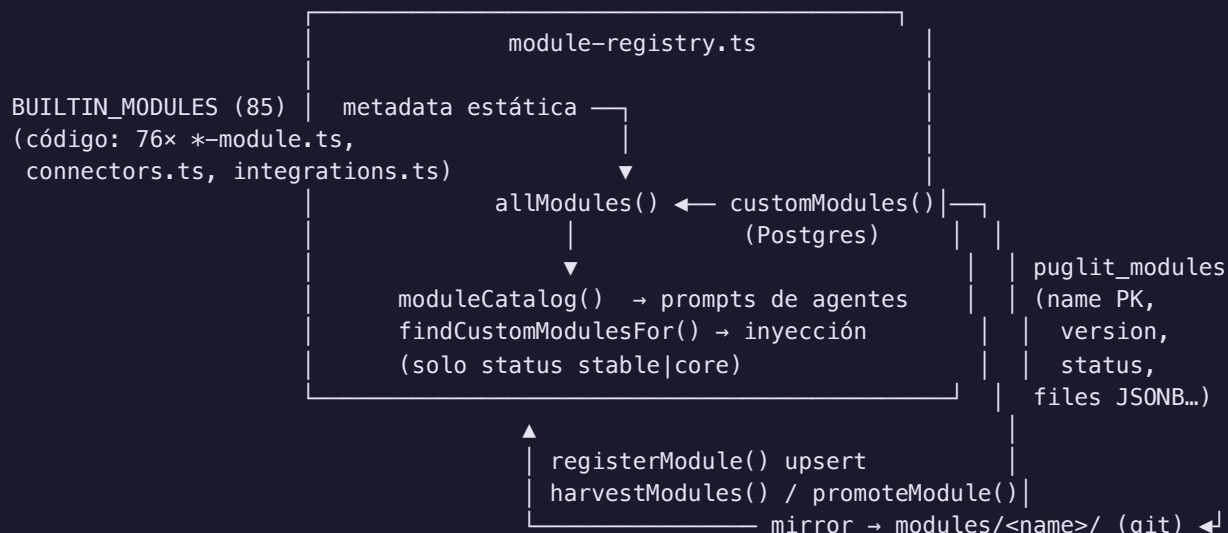
Module	Requiere	Razón
social-auth	crypto	cifrar tokens OAuth en reposo
payments	crypto	tokens/secretos de pago cifrados
billing	crypto	ídem para suscripciones
inappnotify	realtime	publicar notificaciones en vivo (SSE)
moderation	llm	clasificador de toxicidad/NSFW
rag	llm	generación sobre el contexto recuperado
agent	llm, crypto	razonamiento + secretos de tools
charts	stats	capa matemática de los dashboards
forms	validation	mismas reglas en form y API
webhooksout	crypto, queue	firma HMAC + reintentos durables
imagegen	storage	persistir las imágenes generadas
media	storage	persistir imágenes procesadas
graphify	llm	extracción de triples por LLM

Pairings semánticos (declarados en las descripciones, no como deps duras): docparse → rag / llm (file→text→AI); ocr → docs; memorygraph + agent + llm; inventory graduando a wms; pos decrementa inventory si está presente; entitlements = billing × featureflags; cloudflare se empareja con el rate-limit del spine + cache.

Gateways (sidecars Docker requeridos). Muchos integration modules necesitan un servicio externo OSS corriendo: scrape / scraper / pdf / media / docparse / ocr / docgen → el scraper-server; nango, n8n, whatsapp (OpenWA), apprise, crm (EspoCRM), helpdesk (Chatwoot), marketing (Mautic), esign (Documenso), ecommerce (Medusa), sso (Keycloak), bi (Metabase), etc. La metadata gateway se muestra en el catálogo como [needs <gateway>] para que el swarm sepa que esa capacidad implica infra.

10. Registry Design

El registry es deliberadamente **híbrido**: builtins en código (rápidos, versionados, testeados) + customs en Postgres (dinámicos, evolucionables).



Funciones públicas del registry y su rol:

- `customModules()` — lee `puglit_modules` de Postgres. Si falla, devuelve `[]` (degradación elegante).
- `allModules()` — merge builtins + customs, con override por nombre.
- `moduleCatalog()` — serializa todo a texto para inyectar en prompts (los agentes VEN).
- `findCustomModulesFor(text)` — filtra customs `stable` / `core` cuyo vocabulario matchea el producto (los agentes REUSAN).
- `promoteModule(name, to)` — avanza el lifecycle.
- `registerModule(m)` — upsert con bump de versión + espejo a git (los agentes EXTIENDEN/SANAN).
- `harvestModules(files)` — cosecha lo nuevo (el directorio CRECE).

La tabla `puglit_modules` (de `web/sql/genetic.sql`) es minimalista y suficiente: `name` PK, `category`, `description`, `when_to_use`, `env_vars` JSONB, `deps` JSONB, `gateway`, `files` JSONB (el código completo), `version`, `status`, `created_by`, timestamps. El campo `files` JSONB NOT NULL es lo que permite que un module *vivido* persista íntegro y se rehidrate en cualquier futuro build.

La decisión de mantener los builtins **en código y no en la tabla** es intencional: son el genoma estable, deben versionarse con el repo, pasar code review humano y testearse en CI. Solo el genoma *evolucionado* (lo que el swarm inventa) vive en la base mutable.

11. Module Quality Metrics

Puglit mide los modules por **evidencia**, no por declaración. Las señales de calidad que gobiernan promoción y reuso:

- **status como proxy de confianza.** Un module solo es inyectable a builds reales si es `stable` / `core` (`findCustomModulesFor`). El gate `experimental` → `candidate` → `stable` es la métrica de calidad operativa: un module solo sube si acumula evidencia de funcionar.
- **version como historial de mejora.** Una versión alta indica un module muy iterado; el upsert con `status='improved'` deja rastro de cada refinamiento.

- **Tests de dominio embebidos.** Los modules correctness-critical **traen sus propios tests vitest**. `rentals-module.ts` ships `lib/rentals/__tests__/domain.test.ts` con aserciones exactas sobre pricing (`total_cents` debe ser `39600` para un caso dado, todo entero), refunds por política, y solapamiento half-open. Estos tests corren en el runtime gate (`build-local.mjs`) y aportan **cobertura real** — no cobertura inflada.
- **El runtime gate como medida final.** Ningún module se considera "bueno" porque compile. El gate boota la app, hace smoke test de páginas y exige **0 respuestas 5xx**, más una *Queen evidence review* que mide cobertura real y rebota builds por debajo de la barra. La métrica de calidad de un module es, en última instancia, *¿la app que lo usa arranca y funciona?*
- **Trazabilidad (`created_by`).** Cada custom module registra qué agente/proyecto lo originó, habilitando atribución de calidad a equipos genéticos.

La filosofía: la calidad no se afirma, se **gana cruzando gates**. Un module probado en muchos builds que cruza el runtime gate consistentemente sube a `stable` y luego a `core` . Uno que rompe builds se queda en cuarentena o se deprecia.

12. Module Statistics

Conteo real, verificado contra `BUILTIN_MODULES` en `web/lib/module-registry.ts` : **85 builtin modules**, distribuidos en cuatro categorías.

Category	Conteo	Rol en el genoma
<code>util</code>	44	Primitivas Postgres-native zero-dep
<code>integration</code>	26	Conexión a servicios externos / SaaS / gateways
<code>agent</code>	9	Capa de inteligencia (IA, voz, RAG, memoria)
<code>channel</code>	7	Mensajería bidireccional
Total	85	

(Verificación: `grep -oE 'category: "[a-z]+'` sobre el archivo → 44 util + 26 integration + 9 agent + 7 channel = 85. Implementaciones: 76 archivos `web/lib/*-module.ts` , más `connectors.ts` / `integrations.ts` para los builtins de mensajería.)

channel (7) — mensajería bidireccional

`telegram` , `email` , `whatsapp` , `slack` , `discord` , `teams` , y los connectors harvested se categorizan aquí. Cubren Bot API, IMAP/SMTP universal, OpenWA gateway, Events API de Slack, webhooks de Discord/Teams.

agent (9) — capa de inteligencia

`voice` (STT+TTS provider-agnostic), `llm` (cliente OpenAI-compatible, Ollama local por defecto), `rag` (pgvector nativo), `docparse` (markdown), `ocr` (Tesseract), `memorygraph` (knowledge-graph memory), `obsidian` (vault PKM), `graphify` (texto→knowledge graph), `agent` (el cerebro JARVIS, omnicanal con tool-calling y memoria persistente).

integration (26) — servicios externos / SaaS / gateways

`scrape`, `nango` (OAuth a 100s de SaaS), `n8n` (400+ integraciones), `payments` (Stripe/MercadoPago/PayU por país), `social-auth`, `billing`, `cloudflare`, `sms` (Twilio), `webhooksout`, `api` (builder REST+OpenAPI), `shipping` (Shippo), `flights`, `crm` (EspoCRM), `helpdesk` (Chatwoot), `marketing` (Mautic), `knowledgebase` (Outline), `esign` (Documenso), `ecommerce` (Medusa), `pim` (Akeneo), `wms` (OpenBoxes), `invoicing` (Invoice Ninja), `productanalytics` (PostHog), `sso` (Keycloak), `bi` (Metabase), `projectmgmt` (Plane), `dms` (Paperless-ngx).

util (44) — primitivas Postgres-native zero-dep

El bloque más grande, casi todo zero-dependency sobre Postgres: `apprise`, `crypto`, `scraper`, `maps`, `storage`, `queue` (jobs durables FOR UPDATE SKIP LOCKED), `search` (FTS), `realtime` (SSE), `pdf`, `media`, `imagegen`, `cache`, `twofa`, `push`, `socialsearch`, `forecast`, `compress`, `featureflags`, `auditlog` (hash-chained), `booking`, `reviews`, `comments`, `inappnotify`, `referrals`, `moderation`, `multitenancy`, `seo`, `docgen`, `entitlements`, `errortracking`, `migrations`, `wallet` (ledger append-only), `validation`, `forms`, `admin`, `inventory`, `pos`, `stats`, `charts`, `observability`, `uptime`, `statuspage`, `logs`, `rentals` (el vertical Airbnb/booking marketplace).

Verticales destacados. Tres modules encapsulan dominios completos correctness-critical: `rentals` (booking marketplace — pricing determinístico, anti-double-booking por `EXCLUDE USING gist`, refund-by-policy-snapshot, double-blind reviews, todo parametrizado a la tabla detectada), `wallet` (ledger append-only donde `balance = SUM(ledger)` y nunca puede driftar), y `rag` (búsqueda semántica pgvector con SQL guardado). Son la prueba de que un "gen" puede portar lógica de negocio entera, no solo un wrapper de API.

Cierre. El Module Genome es lo que separa a Puglit de un generador de código que escupe LLM crudo. Cada capacidad reutilizable es un gen: metadata que el swarm razona, código determinístico que se inyecta byte-por-byte, dependencias que se resuelven por clausura transitiva, SQL idempotente y guardado, un lifecycle con gates de calidad, y un mecanismo de harvest que hace que el genoma **crezca con cada build**. 85 builtins hoy; mañana, lo que el swarm haya cosechado y promovido. El directorio es vivo porque el organismo aprende.